

C05-Assesment Tool-02

CSA0607-Design and Analysis of Algorithms

Name:A.Sai Joshitha(192424220)

Q11. Traveling Salesman Problem (Backtracking / NP-Hard Problem) Design a pseudocode using Backtracking to find the minimum cost tour that visits all cities exactly once and returns to the starting city. Clearly identify inputs (cost matrix, number of cities) and output (minimum cost path). Develop a logically correct algorithm that explores all possible tours while maintaining path validity. Use appropriate control structures such as loops and recursive calls for permutation generation. Incorporate pruning techniques such as cost bounding to eliminate non-optimal paths. Ensure the pseudocode is well-structured and easy to follow. Handle all possible cases and optimize the solution efficiently.

AIM

To design and implement a Backtracking algorithm for the Traveling Salesman Problem (TSP) to find the minimum-cost tour that visits every city exactly once and returns to the starting city, using cost bounding and pruning to improve the efficiency of the solution.

1. PROBLEM STATEMENT

The Traveling Salesman Problem (TSP) is a combinatorial optimization problem in which a salesman has to visit a set of cities exactly once and return to the starting city while minimizing the total travel cost.

Given a cost matrix representing the travel cost between cities, the objective is to find the tour with the minimum possible total cost.

The algorithm uses Backtracking to generate possible tours and cost bounding to eliminate branches that cannot produce a better solution.

2. OBJECTIVES

- To understand the Traveling Salesman Problem.
- To implement TSP using Backtracking.

- To generate possible city permutations using recursion.
- To ensure every city is visited exactly once.
- To ensure the salesman returns to the starting city.
- To calculate the total cost of each valid tour.
- To use cost bounding for pruning.
- To find the minimum-cost tour.
- To analyze the time and space complexity.

3. INPUT

The algorithm takes the following inputs:

1. Cost Matrix

- Represents the cost of traveling between every pair of cities.

2. Number of Cities

- Represents the total number of cities.

3. Starting City

- Represents the city from which the salesman starts the journey.

Example cost matrix:

From / To	A	B	C	D
A	0	10	15	20
B	10	0	35	25
C	15	35	0	30
D	20	25	30	0

Number of cities: $n = 4$

Starting city: A

4. OUTPUT

The algorithm produces:

- The minimum travel cost
- The optimal tour/path

For the given example:

$$A \rightarrow B \rightarrow D \rightarrow C \rightarrow A$$

Minimum cost:

$$80$$

5. CONSTRAINTS

The following constraints must be satisfied:

5.1 Every City Must Be Visited

Every city must be visited exactly once.

5.2 No City Repetition

A city that has already been visited cannot be selected again.

5.3 Valid Path

There must be a valid route between consecutive cities.

5.4 Return to Starting City

After visiting all cities, the salesman must return to the starting city.

5.5 Minimum Cost

Among all feasible tours, the tour having the smallest total cost must be selected.

6. BACKTRACKING APPROACH

Backtracking solves the problem by constructing the tour one city at a time.

The algorithm performs the following steps:

1. Start from the selected starting city.

2. Mark the starting city as visited.
3. Select an unvisited city.
4. Add the city to the current path.
5. Calculate the current travel cost.
6. Calculate a lower-bound estimate for the remaining cities.
7. If the estimated cost cannot improve the current best cost, prune the branch.
8. Otherwise, recursively continue with the selected city.
9. When all cities are visited, return to the starting city.
10. Compare the complete tour cost with the current minimum.
11. Backtrack and try other possibilities.
12. Finally, return the minimum-cost tour.

7. COST BOUNDING AND PRUNING

Cost bounding is used to reduce unnecessary exploration.

Let: $CurrentCost$

be the cost accumulated so far.

Let: $Bound$

be the estimated minimum additional cost required to complete the tour.

Then: $EstimatedCost = CurrentCost + Bound$

If: $EstimatedCost \geq BestCost$

then the branch cannot produce a better solution.

Therefore, the branch is pruned.

$$\boxed{CurrentCost + Bound \geq BestCost \Rightarrow Prune}$$

This improves the practical efficiency of the Backtracking algorithm.

8. PSEUDOCODE

Main Algorithm

ALGORITHM TSP_Backtracking(cost, n, start)

INPUT:

cost[1..n][1..n] → Cost matrix

n → Number of cities

start → Starting city

OUTPUT:

bestCost → Minimum cost

bestPath → Optimal tour

BEGIN

bestCost $\leftarrow \infty$

bestPath $\leftarrow$ EMPTY

FOR i $\leftarrow$ 1 TO n DO

visited[i] $\leftarrow$ FALSE

END FOR

visited[start] $\leftarrow$ TRUE

path[1] $\leftarrow$ start

CALL Explore(start, 1, 0)

IF bestCost = ∞ THEN

PRINT "No valid tour exists"

ELSE

PRINT "Minimum Cost =", bestCost

PRINT "Optimal Tour =", bestPath

END IF

END

9. RECURSIVE BACKTRACKING PROCEDURE

PROCEDURE Explore(current, level, currentCost)

BEGIN

IF level = n THEN

IF cost[current][start] is valid THEN

totalCost $\leftarrow$ currentCost + cost[current][start]

IF totalCost < bestCost THEN

bestCost $\leftarrow$ totalCost

bestPath $\leftarrow$ path + [start]

END IF

END IF

RETURN

END IF

FOR nextCity $\leftarrow$ 1 TO n DO

IF visited[nextCity] = FALSE AND

cost[current][nextCity] is valid THEN

newCost $\leftarrow$ currentCost +

cost[current][nextCity]

bound $\leftarrow$ CalculateLowerBound(

nextCity,

visited

)

IF newCost + bound < bestCost THEN

```
visited[nextCity] ← TRUE
```

$$\text{path}[\text{level} + 1] \leftarrow \text{nextCity}$$

CALL Explore(

nextCity,

level + 1,

newCost

)

```
visited[nextCity] ← FALSE
```

END IF

END IF

END FOR

END

10. LOWER-BOUND PROCEDURE

FUNCTION CalculateLowerBound(current, visited)

BEGIN

$$\text{bound} \leftarrow 0$$

FOR city $\leftarrow$ 1 TO n DO

IF visited[city] = FALSE THEN

```
minimum ← ∞
```

FOR nextCity $\leftarrow$ 1 TO n DO

IF city $\neq$ nextCity AND

cost[city][nextCity] is valid THEN

```

        IF cost[city][nextCity] < minimum THEN

            minimum  $\leftarrow$  cost[city][nextCity]

        END IF

    END IF

END FOR

IF minimum  $\neq \infty$  THEN

    bound  $\leftarrow$  bound + minimum

END IF

END IF

END FOR

RETURN bound

END

```

11. PYTHON PROGRAM

```

INF = float('inf')

```

```

# Cost matrix

```

```

cost = [

    [0, 10, 15, 20],

    [10, 0, 35, 25],

    [15, 35, 0, 30],

    [20, 25, 30, 0]

]

```

```

n = len(cost)

```

```

# Starting city

```

```

start = 0

```

```

best_cost = INF

best_path = []

visited = [False] * n

path = [start]

def calculate_lower_bound(visited):

    bound = 0

    for city in range(n):

        if not visited[city]:

            minimum = INF

            for next_city in range(n):

                if city != next_city and cost[city][next_city] != INF:

                    if cost[city][next_city] < minimum:

                        minimum = cost[city][next_city]

            if minimum != INF:

                bound += minimum

    return bound

def tsp(current, level, current_cost):

    global best_cost, best_path

    # Base case

    if level == n:

        # Return to starting city

        if cost[current][start] != INF:

            total_cost = current_cost + cost[current][start]

```

```

        if total_cost < best_cost:

            best_cost = total_cost

            best_path = path[:] + [start]

    return

# Try every unvisited city
for next_city in range(n):

    if (not visited[next_city] and

        cost[current][next_city] != INF):

        new_cost = (

            current_cost +

            cost[current][next_city]

        )

        # Calculate lower bound

        bound = calculate_lower_bound(visited)

        # Branch and Bound pruning

        if new_cost + bound >= best_cost:

            continue

        # Choose the city

        visited[next_city] = True

        path.append(next_city)

        # Recursive call

        tsp(

            next_city,

            level + 1,

```

```

        new_cost

    )

    # Backtrack

    path.pop()

    visited[next_city] = False

# Start the algorithm

visited[start] = True

tsp(start, 1, 0)

# Display result

if best_cost == INF:

    print("No valid tour exists")

else:

    print("Minimum Cost:", best_cost)

    print("Optimal Tour:", end=" ")

    for city in best_path:

        print(chr(65 + city), end=" ")

    print()

```

12. OUTPUT

Minimum Cost: 80

Optimal Tour: A B D C A

Therefore:

$$\boxed{\text{Minimum Cost} = 80}$$

and:

$$\boxed{\text{Optimal Tour} = A \rightarrow B \rightarrow D \rightarrow C \rightarrow A}$$

13. WORKING OF THE PROGRAM

The program begins with city A.

Possible first choices are:

$A \rightarrow B$

$A \rightarrow C$

$A \rightarrow D$

The algorithm recursively explores each possible unvisited city.

For example:

$A \rightarrow B \rightarrow C \rightarrow D \rightarrow A$

has cost:

$$10 + 35 + 30 + 20 = 95$$

Another tour is:

$A \rightarrow B \rightarrow D \rightarrow C \rightarrow A$

Its cost is:

$$10 + 25 + 30 + 15 = 80$$

Since 80 is smaller than 95:

$$BestCost = 80$$

The algorithm continues exploring other branches. If the lower-bound estimate of a branch is already greater than or equal to 80, that branch is discarded.

Finally, the minimum-cost tour is obtained.

14. DRY RUN

For the given input:

$A \rightarrow B \rightarrow C \rightarrow D \rightarrow A$

Cost:

$$10 + 35 + 30 + 20 = 95$$

Current best:

95

Next:

$A \rightarrow B \rightarrow D \rightarrow C \rightarrow A$

Cost:

$$10 + 25 + 30 + 15 = 80$$

Update:

$$BestCost = 80$$

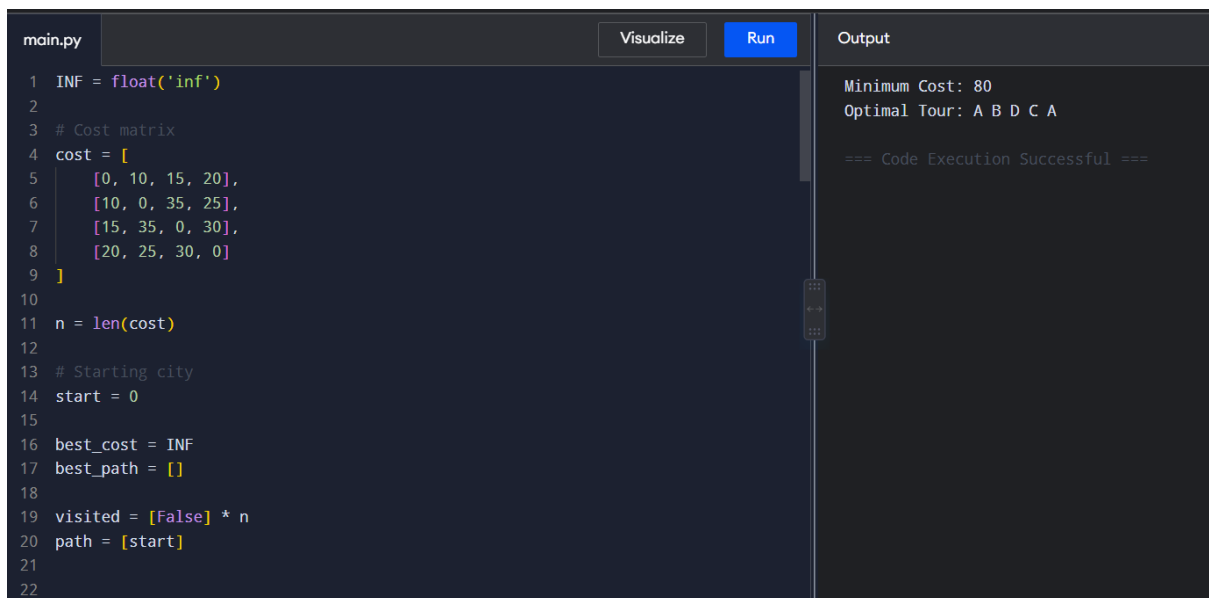
Another branch that has an estimated cost greater than or equal to 80 is pruned.

The final answer is:

$A \rightarrow B \rightarrow D \rightarrow C \rightarrow A$

with:

Minimum Cost = 80

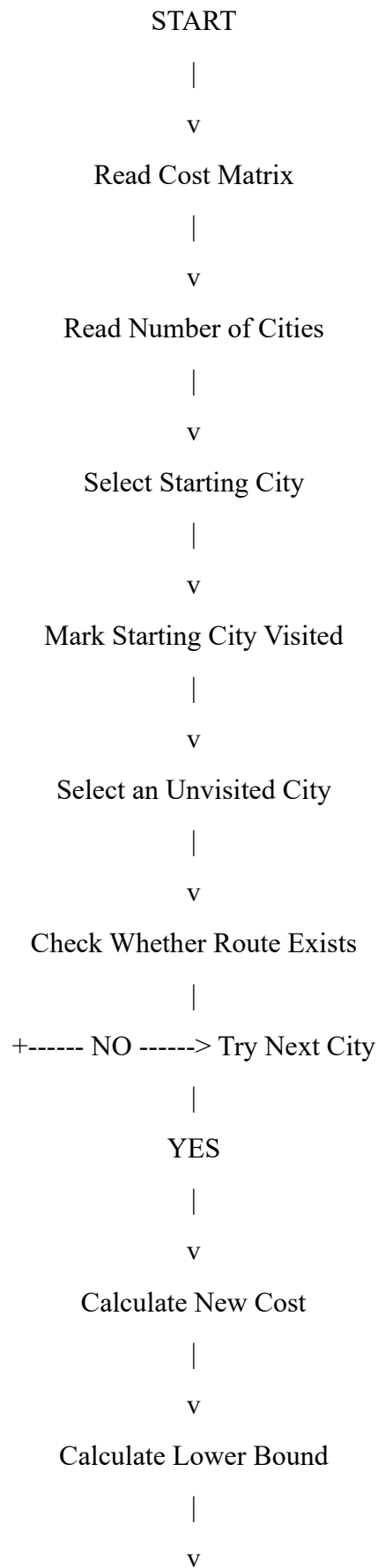


```
main.py Visualize Run Output
1 INF = float('inf')
2
3 # Cost matrix
4 cost = [
5     [0, 10, 15, 20],
6     [10, 0, 35, 25],
7     [15, 35, 0, 30],
8     [20, 25, 30, 0]
9 ]
10
11 n = len(cost)
12
13 # Starting city
14 start = 0
15
16 best_cost = INF
17 best_path = []
18
19 visited = [False] * n
20 path = [start]
21
22
```

Minimum Cost: 80
Optimal Tour: A B D C A

=== Code Execution Successful ===

15. ALGORITHM FLOW



Is Estimated Cost < Best Cost?

|

+----- NO -----> PRUNE

|

YES

|

v

Mark City Visited

|

v

Recursive Call

|

v

All Cities Visited?

|

+----- NO -----> Continue

|

YES

|

v

Return to Starting City

|

v

Calculate Total Cost

|

v

Update Best Solution

|

v

BACKTRACK

|

v

Display Minimum Cost Tour

|

v

END

16. CORRECTNESS OF THE ALGORITHM

The algorithm is correct for the following reasons:

City Visitation

The visited array ensures that each city is selected only once.

Path Validity

A city is added only when a valid route exists between the current city and the selected city.

Complete Tour

When all cities are visited, the algorithm checks the route back to the starting city.

Minimum Cost

Every feasible complete tour is compared with the current best_cost.

Safe Pruning

A branch is pruned only when:

$$CurrentCost + Bound \geq BestCost$$

Therefore, the branch cannot provide a better solution.

Hence, the algorithm produces an optimal tour.

17. TIME COMPLEXITY

For n cities, the number of possible tours is approximately:

$$(n - 1)!$$

Therefore, the worst-case time complexity is:

$$O((n - 1)!)$$

It is commonly represented as:

$$O(n!)$$

Although Branch and Bound can reduce the number of branches explored in practice, the worst-case complexity remains factorial.

18. SPACE COMPLEXITY

The algorithm requires space for:

- Visited array
- Current path
- Recursion stack
- Best path

Therefore, auxiliary space is:

$$O(n)$$

The cost matrix requires:

$$O(n^2)$$

space.

19. SPECIAL CASES

The algorithm can handle the following cases:

Case 1: One City

If there is only one city, the salesman starts and returns to that city.

Case 2: No Valid Route

If there is no possible route connecting all cities into a complete tour, the program displays:

No valid tour exists

Case 3: Unavailable Route

An unavailable route can be represented using:

Such routes are ignored during the search.

Case 4: Multiple Optimal Tours

If multiple tours have the same minimum cost, any one of them can be considered an optimal solution.

20. ADVANTAGES

- Finds the exact optimal solution.
- Easy to understand and implement.
- Uses recursion to generate possible tours.
- Prevents repeated city visits.
- Uses pruning to reduce unnecessary searches.
- Suitable for small TSP problems.
- Demonstrates Backtracking and Branch and Bound concepts.

21. LIMITATIONS

- Worst-case time complexity is factorial.
- Becomes slow as the number of cities increases.
- Large problems may require considerable computation.
- Exact solutions are difficult for very large datasets.
- Performance depends on the effectiveness of the bounding technique.

22. APPLICATIONS

The Traveling Salesman Problem can be applied to:

- Delivery route optimization
- Logistics
- Transportation planning
- Vehicle routing
- Robot navigation

- Drone route planning
- Tourist route planning
- Warehouse operations
- Manufacturing
- Network optimization

23. RESULT

The Traveling Salesman Problem was successfully implemented using Backtracking with cost bounding.

For the given cost matrix, the algorithm obtained:

$$\boxed{\textit{Optimal Tour} = A \rightarrow B \rightarrow D \rightarrow C \rightarrow A}$$

and:

$$\boxed{\textit{Minimum Cost} = 80}$$

The algorithm successfully ensures that all cities are visited exactly once, the salesman returns to the starting city, and costly non-promising branches are pruned.

24. CONCLUSION

The Traveling Salesman Problem was solved using a recursive Backtracking algorithm with cost bounding. The algorithm generates possible tours while maintaining path validity and preventing repeated city visits.

The Branch and Bound technique improves efficiency by eliminating branches whose estimated cost cannot produce a better solution. Since only non-promising branches are removed, the optimal solution is preserved.

Although the approach provides an exact solution, its worst-case time complexity is:

$$\boxed{O(n!)}$$

Therefore, it is suitable mainly for small and moderate-sized TSP instances.